Library Management System

SQL Portfolio Project: Complete Project Details

Item	Detail
Database	SQLite (portable to MySQL / PostgreSQL)
Domain	Library / school management
Tables	7 tables, 1 view, 3 indexes
Sample data	25 members, 30 books, 15 authors, 8 categories, 220 loans, 64 fines
Loan period covered	January to October 2026
Analysis queries	13
Files	01_schema.sql, 02_seed_data.sql, 03_analysis_queries.sql, gen_seed.py, README.md

1. Project Overview

This project models the day-to-day operations of a library and uses SQL to answer the questions a librarian or manager would actually ask: which books are popular, which loans are overdue, who owes fines, and how borrowing changes month to month.

Objectives

• Design a normalised relational schema with proper keys and constraints.
• Populate it with realistic, reproducible sample data.
• Write analysis queries that go from basic joins to window functions and CTEs.
• Turn query results into business findings.

Tools used

SQLite for the database, Python (standard library) to generate the seed data with a fixed random seed, so every run produces identical results.

2. Database Design

The schema has seven tables. **book_authors** is a bridge table that allows a book to have several authors and an author to have several books. **loans** records every borrowing; a NULL *returned_on* means the book is still out. **fines** holds one fine per late loan.

Relationships

Foreign key	References	Meaning
books.category_id	categories.category_id	Many books to one category
book_authors.book_id	books.book_id	Bridge: book side
book_authors.author_id	authors.author_id	Bridge: author side
loans.book_id	books.book_id	Many loans to one book
loans.member_id	members.member_id	Many loans to one member
fines.loan_id	loans.loan_id	At most one fine per loan (UNIQUE)

Table: members

Column	Type	Constraints
member_id	INTEGER	PK
full_name	TEXT	NOT NULL
email	TEXT	NOT NULL
member_type	TEXT	NOT NULL
joined_on	DATE	NOT NULL

Table: categories

Column	Type	Constraints
category_id	INTEGER	PK
name	TEXT	NOT NULL

Table: authors

Column	Type	Constraints
author_id	INTEGER	PK
full_name	TEXT	NOT NULL
country	TEXT	

Table: books

Column	Type	Constraints
book_id	INTEGER	PK
title	TEXT	NOT NULL
category_id	INTEGER	FK -> categories.category_id, NOT NULL
published_year	INTEGER	
copies_total	INTEGER	NOT NULL, DEFAULT 1

Table: book_authors

Column	Type	Constraints
book_id	INTEGER	PK, FK -> books.book_id
author_id	INTEGER	PK, FK -> authors.author_id

Table: loans

Column	Type	Constraints
loan_id	INTEGER	PK
book_id	INTEGER	FK -> books.book_id, NOT NULL
member_id	INTEGER	FK -> members.member_id, NOT NULL
borrowed_on	DATE	NOT NULL
due_on	DATE	NOT NULL
returned_on	DATE	

Table: fines

Column	Type	Constraints
fine_id	INTEGER	PK
loan_id	INTEGER	FK -> loans.loan_id, NOT NULL
amount	REAL	NOT NULL
paid	INTEGER	NOT NULL, DEFAULT 0

Integrity rules (CHECK constraints)

- member_type must be student, teacher or public
- published_year between 1400 and 2100; copies_total >= 0
- due date cannot be before the borrow date; return date cannot be before the borrow date
- fine amount >= 0; paid flag is 0 or 1
- Foreign keys are enforced (PRAGMA foreign_keys = ON)

Indexes and view

Indexes on loans(member_id), loans(book_id) and books(category_id) speed up the joins used by most queries. The view **v_active_loans** lists every loan that has not been returned, with the member name and book title.

3. Sample Data

The data is generated by **gen_seed.py** using `random.seed(42)`, so it is fully reproducible. Row counts are below.

Table	Rows
members	25
books	30
authors	15
categories	8
book_authors	30
loans	220
fines	64

Design choices in the data

• Popularity weights make some books (e.g. Atomic Habits) borrowed far more than others, so rankings are interesting rather than flat.
• About 70% of returns are on time and 30% late; late returns generate a fine at 2.00 per day.
• Recent loans may still be out, which creates currently overdue cases.
• Member mix is mostly students, with some teachers and public members.

Books per category

Category	Books
Fiction	13
Science	4
Fantasy	4
History	3
Technology	2
Mathematics	2
Biography	1
Self-Help	1

4. Analysis Queries and Results

Each query is shown with its business question, the SQL, and the output produced on the sample data (first 10 rows shown). The reference date used for overdue checks is 2026-10-17.

Q1. Most borrowed books (top 10)

Question: Which books are borrowed most? Helps decide which titles need more copies.

```
SELECT b.title, COUNT(*) AS times_borrowed
FROM loans l
JOIN books b ON b.book_id = l.book_id
GROUP BY b.book_id
ORDER BY times_borrowed DESC, b.title
LIMIT 10;
```

title	times_borrowed
Atomic Habits	24
Things Fall Apart	14
Discrete Mathematics Primer	13
I Am Malala	12
I, Robot	12
The Way of Kings	12
A Brief History of Time	10
Cosmos	10
Sapiens	10
Mistborn	9

Q2. Loans and share of total by category

Question: How are loans distributed across genres, and what share does each hold?

```
SELECT c.name AS category,
       COUNT(*) AS loans,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct_of_loans
FROM loans l
JOIN books b      ON b.book_id = l.book_id
JOIN categories c ON c.category_id = b.category_id
GROUP BY c.category_id
ORDER BY loans DESC;
```

category	loans	pct_of_loans
Fiction	73	33.2
Fantasy	32	14.5
Science	27	12.3
Self-Help	24	10.9
History	23	10.5
Mathematics	19	8.6
Biography	12	5.5
Technology	10	4.5

Q3. Currently overdue loans (not returned, past due date)

Question: Which borrowed books are past their due date right now, and by how many days?

```
SELECT m.full_name, b.title, l.due_on,
       CAST(julianday('2026-10-17') - julianday(l.due_on) AS INTEGER) AS days_overdue
FROM loans l
JOIN members m ON m.member_id = l.member_id
JOIN books b   ON b.book_id   = l.book_id
WHERE l.returned_on IS NULL
  AND l.due_on < '2026-10-17'
ORDER BY days_overdue DESC;
```

full_name	title	due_on	days_overdue
Neel Sharma	Contact	2026-10-15	2
Rahul Iyer	Contact	2026-10-16	1

Q4. Books with no copies currently available

Question: Which titles have every copy checked out at the moment?

```
SELECT b.title, b.copies_total, COUNT(l.loan_id) AS currently_out
FROM books b
LEFT JOIN loans l ON l.book_id = b.book_id AND l.returned_on IS NULL
GROUP BY b.book_id
HAVING currently_out >= b.copies_total
ORDER BY b.title;
```

title	copies_total	currently_out
Contact	2	2

Q5. Top 5 most active members

Question: Who are the most active borrowers?

```
SELECT m.full_name, m.member_type, COUNT(*) AS loans
FROM loans l
JOIN members m ON m.member_id = l.member_id
GROUP BY m.member_id
ORDER BY loans DESC, m.full_name
LIMIT 5;
```

full_name	member_type	loans
Kabir Reddy	public	14
Arjun Gupta	student	13
Isha Nair	student	13
Zoya Nair	student	13
Dev Reddy	teacher	12

Q6. Monthly borrowing trend with month-over-month change (window function)

Question: How does monthly borrowing change over time? Uses a CTE and LAG().

```
WITH monthly AS (
    SELECT strftime('%Y-%m', borrowed_on) AS month, COUNT(*) AS loans
    FROM loans
    GROUP BY month
)
SELECT month, loans,
       loans - LAG(loans) OVER (ORDER BY month) AS change_vs_prev
FROM monthly
ORDER BY month;
```

month	loans	change_vs_prev
2026-01	23	

month	loans	change_vs_prev
2026-02	19	-4
2026-03	20	1
2026-04	24	4
2026-05	30	6
2026-06	27	-3
2026-07	21	-6
2026-08	19	-2
2026-09	17	-2
2026-10	20	3

Q7. Fines summary by member type

Question: How much fine money has been charged and how much is unpaid, by member type?

```
SELECT m.member_type,
       COUNT(f.fine_id)                              AS fines_count,
       ROUND(SUM(f.amount), 2)                       AS total_fines,
       ROUND(SUM(CASE WHEN f.paid = 0 THEN f.amount ELSE 0 END), 2) AS unpaid
FROM fines f
JOIN loans l   ON l.loan_id = f.loan_id
JOIN members m ON m.member_id = l.member_id
GROUP BY m.member_type
ORDER BY total_fines DESC;
```

member_type	fines_count	total_fines	unpaid
student	39	976.0	508.0
public	24	540.0	334.0
teacher	1	4.0	0.0

Q8. Late-return rate per category

Question: Which genres are returned late most often?

```
SELECT c.name AS category,
       COUNT(*) AS returned_loans,
       ROUND(100.0 * SUM(l.returned_on > l.due_on) / COUNT(*), 1) AS late_pct
FROM loans l
JOIN books b      ON b.book_id = l.book_id
JOIN categories c ON c.category_id = b.category_id
WHERE l.returned_on IS NOT NULL
GROUP BY c.category_id
ORDER BY late_pct DESC;
```

category	returned_loans	late_pct
Mathematics	18	38.9
Self-Help	23	34.8
Science	27	33.3
Fiction	66	33.3
Biography	11	27.3
History	23	26.1
Fantasy	31	25.8
Technology	9	11.1

Q9. Rank books within each category by popularity (RANK window)

Question: Top two books inside each category using RANK() with PARTITION BY.

```
SELECT category, title, loans, rnk
FROM (
    SELECT c.name AS category, b.title, COUNT(l.loan_id) AS loans,
           RANK() OVER (PARTITION BY c.category_id ORDER BY COUNT(l.loan_id) DESC) AS rnk
    FROM books b
    JOIN categories c ON c.category_id = b.category_id
    LEFT JOIN loans l ON l.book_id = b.book_id
    GROUP BY b.book_id
)
WHERE rnk <= 2
ORDER BY category, rnk, title;
```

category	title	loans	rnk
Biography	I Am Malala	12	1
Fantasy	The Way of Kings	12	1
Fantasy	Mistborn	9	2
Fiction	Things Fall Apart	14	1
Fiction	I, Robot	12	2
History	Sapiens	10	1
History	21 Lessons for the 21st Century	7	2
Mathematics	Discrete Mathematics Primer	13	1
Mathematics	Calculus Made Easy	6	2
Science	A Brief History of Time	10	1

Showing 10 of 14 rows.

Q10. Most borrowed authors (via many-to-many bridge)

Question: Which authors are borrowed most, through the many-to-many bridge table?

```
SELECT a.full_name, a.country, COUNT(l.loan_id) AS loans
FROM authors a
JOIN book_authors ba ON ba.author_id = a.author_id
JOIN loans l         ON l.book_id = ba.book_id
GROUP BY a.author_id
ORDER BY loans DESC
LIMIT 5;
```

full_name	country	loans
Brandon Sanderson	USA	27
Isaac Asimov	USA	26
James Clear	USA	24
Donald Knuth	USA	23
Yuval Noah Harari	Israel	23

Q11. Members who never borrowed anything

Question: Are there members who joined but never borrowed anything?

```
SELECT m.full_name, m.joined_on
FROM members m
WHERE NOT EXISTS (SELECT 1 FROM loans l WHERE l.member_id = m.member_id);
```

Result: 0 rows. Every member has borrowed at least one book.

Q12. Average loan duration (days) by member type, returned loans only

Question: How long do different member types keep a book on average?

```
SELECT m.member_type,
       ROUND(AVG(julianday(l.returned_on) - julianday(l.borrowed_on)), 1) AS avg_days_kept
FROM loans l
JOIN members m ON m.member_id = l.member_id
WHERE l.returned_on IS NOT NULL
GROUP BY m.member_type;
```

member_type	avg_days_kept
public	13.8
student	14.7
teacher	8.6

Q13. Active loans via the view

Question: Quick look at current loans using the v_active_loans view.

```
SELECT * FROM v_active_loans ORDER BY due_on LIMIT 10;
```

loan_id	member	title	borrowed_on	due_on
179	Neel Sharma	Contact	2026-10-01	2026-10-15
70	Rahul Iyer	Contact	2026-10-02	2026-10-16
11	Zoya Nair	Too Like the Lightning	2026-10-03	2026-10-17
210	Anika Khan	I, Robot	2026-10-04	2026-10-18
42	Ishaan Gupta	Calculus Made Easy	2026-10-05	2026-10-19
159	Aarav Sharma	The Ministry of Utmost Happiness	2026-10-05	2026-10-19
171	Zoya Nair	I Am Malala	2026-10-06	2026-10-20
127	Anika Khan	Atomic Habits	2026-10-09	2026-10-23
157	Rahul Iyer	The Art of Computer Programming	2026-10-09	2026-10-23
53	Pooja Patel	The Blue Umbrella	2026-10-11	2026-10-25

5. Key Findings

• **Atomic Habits** is the most borrowed book with 24 loans.
• Fiction accounts for 33.2% of all loans, the largest share of any category.
• 2 loans are currently overdue; at least one title has no copies available.
• Total fines charged: 1520.00, of which 842.00 is unpaid.
• Brandon Sanderson is the most borrowed author (27 loans).

Recommendations a librarian could act on

• Buy extra copies of the most borrowed titles and of any title that is fully checked out.
• Send reminders for overdue loans and unpaid fines.
• Stock more of the genres that dominate loans, and review the ones that rarely move.

6. SQL Skills Demonstrated

Skill	Where used
INNER / LEFT JOIN, multi-table joins	Q1 to Q13
Many-to-many modelling	book_authors bridge, Q10
GROUP BY, HAVING, aggregates	Q1, Q2, Q4, Q5, Q7, Q8
Window functions (RANK, LAG, SUM OVER)	Q2, Q6, Q9
CTE (WITH)	Q6
Subqueries, NOT EXISTS	Q9, Q11
Date arithmetic and formatting	Q3, Q6, Q12
Views, indexes, CHECK constraints, foreign keys	Schema file

7. How to Run

```
sqlite3 library.db < 01_schema.sql
sqlite3 library.db < 02_seed_data.sql
sqlite3 -header -column library.db < 03_analysis_queries.sql
```

No sqlite3 command-line tool? Run the same files with Python's built-in sqlite3 module (executescript), or paste them into an online SQLite playground. For MySQL or PostgreSQL, change AUTOINCREMENT to AUTO_INCREMENT or SERIAL, remove the PRAGMA line, and replace julianday() and strftime() with the target database's date functions.

8. Ideas to Extend

• Add a reservations table with a waiting-list query.
• Add a trigger that creates a fine automatically when a late book is returned.
• Add a stored procedure for issuing and returning books.
• Connect a BI tool such as Power BI or Tableau to build a dashboard.
• Port the project to PostgreSQL and compare query plans using EXPLAIN.